



۸۱۰۱۰۱۴۹۲



نوت ۶ مهر

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

ادامه فصل اول از اسلاید ۲۴ تا ۵۵ [۱]

سوئیچینگ یک روش است تا پکت را از پورت ورودی به پورت خروجی، *forward* کند.

انواع شبکه از نظر سوئیچینگ

- شبکه‌های سوئیچینگ بسته‌ای *packed – switched* که خود شامل دو نوع شبکه‌های دیتا گرام و شبکه‌های *virtual circuit = VC* هستند.

- شبکه‌های سوئیچینگ مداری که خود شامل دو نوع *FDM = frequency division multiplexing* و *TDM = time division multiplexing* هستند.

داستان *connection – oriented* و *connectionless* بودن، از داستان *datagram* جداست. (در واقع اینطوری که خودم سرچ کردم، *connection based* بودن یعنی یک *handshake* بین فرستنده و گیرنده رخ می‌دهد و یک ارتباط *reliable* رخ می‌دهد. اما در روش *connectionless*، هیچ ارتباط اولیه شکل نمی‌گیرد و فقط داده ارسال می‌شود و تضمینی نیست که داده به مقصد رسیده یانه)

پروتکل‌های *TCP = transmission control protocol* از سرویس *connection – oriented* و پروتکل‌های *UDP = user datagram protocol* از سرویس *connectionless* استفاده می‌کند.

هسته شبکه

هسته شبکه تعداد زیادی روتر متصل به هم هستند که شبیه یک گراف یا مش بنظر می‌آیند.

انواع پیاده‌سازی سوئیچینگ بسته‌ای

- روش دیتاگرام *datagram*
- روش مدار مجازی *virtual circuit = VC*

روش دیتاگرام

در روش دیتاگرام، وقتی داده‌ها را پکت بندی کردیم، روترها به هر پکت به صورت مستقل نگاه می‌کنند و برای‌شان مهم نیست که برای یک داده باشند یا خیر. پس با هر کدوم به طور مستقل و طبق ترافیک هر مسیر شبکه برخورد می‌کنند تا به مقصد برسند. چون پکت‌ها از مسیرهای مختلف به مقصد می‌رسند، تضمینی نیست که به ترتیب برسند، و حتی ممکن است *lost* شوند، و مضاف بر آن، خود مقصد باید پکت‌ها را دوباره مرتب کند.

روش مدار مجازی

وقتی داده‌ها پکت بندی شد، همان اول کار (فاز تعیین مسیر)، مسیر از مبدا به مقصد را مشخص می‌کند و تمام پکت‌های آن داده را از آن مسیر به مقصد هدایت می‌کند. که به این مسیر، مدار می‌گویند. در این روش، فوروارد به پورت خروجی طبق شماره *vc* هر بسته انجام می‌شود. (برعکس روش دیتاگرام که از روی آدرس *IP* به پورت خروجی هدایت می‌شدند). چون این لینک‌های تشکیل دهنده یک مسیر اختصاصی نیستند و داده‌های دیگر هم می‌توانند از آن مسیرها استفاده کنند، پس برای همین به آن مدار مجازی *virtual circuit* می‌گویند. با اینکه همه بسته‌ها از یک مدار عبور می‌کنند ولی ممکن است توسط بسته‌های داده‌های دیگر، بینشان فاصله بیفتد. مقصد هم این بسته‌ها را به اپلیکیشنش *deliver* می‌کند. بسته‌ها وقتی دارند از یک لینک مدار وارد لینک دیگر می‌شوند، علاوه بر اینکه از یک پورت ورودی به پورت خروجی متفاوت با شماره متفاوت می‌روند، شماره *vc* اون بسته‌ها هم تغییر می‌کند. این تغییر شماره پورت و عدد *vc* بسته‌ها در هر روتر از لینک ورودی به لینک خروجی، از روی جدول *vc* روترها که در محله فاز یافتن مدار مشخص شدند، انجام می‌شود.

مقایسه بین دو روش گفته شده

برای داده‌ها طولانی، روش *vc* به کمک سرویس *sequencing* ترتیب بسته‌ها و سرویس *error control* صحت بسته‌ها را هم تضمین می‌کند.

در روش *vc*، چون در همان اول کار مدار مشخص شده‌است، پکت‌ها سریع‌تر به مقصد می‌رسد و لازم نیست در هر روتر، دوباره مسیریابی کند. هزینه مسیریابی اولیه زیاد است اما اگر تعداد بسته‌ها زیاد باشد، بین آنها سر شکن می‌شود و از هزینه مسیریابی برای تک تک بسته‌ها (روش دیتاگرام) کمتر می‌شود.

اما عیب *vc* این است که اگر یکی از لینک‌های مدار از بین برود، کل مسیر از بین می‌رود و به مقصد نمی‌رسد. برای بسته‌های کوتاه، در روش دیتاگرام، چون لازم نیست هزینه زیادی اول کار برای پیدا کردن مدار بدهد، هزینه کمتر و سریع‌تر است. دیتا گرام پیاده سازی ساده‌ای دارد و قابل فهم‌ترست و برای همین اطمینان پذیری خوبی دارد.

در ضمن وقتی یک لینک خراب می‌شود به صورت داینامیک لینک و مسیر جدید انتخاب می‌شود و با احتمال بیشتری به مقصد می‌رسد.

مفهوم *store and forward*

در شبکه‌های پکت سویچینگ، ابتدا باید کل بسته توسط روتر ذخیره شود و بعد با کمک *IP* در دیتاگرام و یا عدد *vc* برای روش *vc* آن را هدایت کند.

برای یافتن پورت خروجی در روش *vc* چون مدار مشخص شده، از روی *exact match* پورت خروجی را تشخیص می‌دهد. در صورتی که در روش دیتاگرام، با کمک *prefix match* (احتمالا از روی *net id* و با مسیریابی) پورت خروجی را تشخیص می‌دهد.

در مثال گفته شده در اسلاید، چون مسیر دو لینک داشت، پس دو بار *transistion delay* رخ خواهد داد که برابر خواهد بود با $2L/R$. (توجه کن که موقع ورود به لینک تاخیر داریم و تاخیر موقع خروج از لینک با همان *propagation delay* حساب می‌شود)

فرق k در حجم داده و مخابرات

در موارد حجم داده و کامپیوتر، یک k برابر با ۱۰۲۴ است اما در موارد سرعت خط و مخابرات، یک k برابر با ۱۰۰۰ است.

پس مثلا وقتی می‌گوییم که L همان طول دیتا، $7.5Mbits$ است یعنی $7.5 \times 1024 \times 1024 \text{ bit}$ ولی وقتی می‌گوییم که $transmission \text{ rate} = R = 1.5Mbps$ یعنی $1.5 \times 1000 \times 1000 \text{ (b/s)}$

در اسلاید، در مسیری که ۲ تا روتر میانی داشتیم و ۵۰۰۰ تا بسته، و هر بسته با تاخیر ۱ میلی‌ثانیه، در کل، رسیدن آخرین بسته به مقصد، ۵۰۰۲ میلی‌ثانیه زمان گرفت. بسته آخر در ۴۹۹۹ راه افتاد در ۵۰۰۰ به اولین روتر رسید و به ازای هر روتر، چون دوباره باید روی لینک دیگری قرار بگیرد، تاخیر ۱ میلی‌ثانیه اضافه می‌شود. قبلی‌ها به صورت *pipeline* خارج شده بودند. در واقع می‌توان گفت اون ۲ میلی‌ثانیه کمترین تاخیر رسیدن بیت اول بسته اول است (پس در مثال ما در ۳ میلی‌ثانیه، آخرین بیت بسته اول به مقصد می‌رسد).

مفهوم *cut – through*

در روترهای جدید، لازم نیست اول کل پکت را بگیریم و بعد پورت خروجی را تشخیص دهیم. می‌توان از روی چند تا بیت اولیه پکت ورودی هم پورت خروجی را تشخیص داد و بقیه داده را به صورت *on the fly* گذراند. به این شکل تاخیر کمتر می‌شود. یعنی وقتی هدر اون پکت رو خوند، از روی اون هدایت کنه به پورت خروجی و دیگه ذخیره نکنه. البته به شرطی که پکت‌ها در صف منتظر ورود به روتر نمانند.

چون از لینک ورودی به صورت *on the fly* به لینک خروجی منتقل می‌شود، دیگر هزینه *transmission delay* نداریم و تنها همان اول کار، این هزینه برای ورود از *host* به اولین لینک، پرداخت می‌شود. بدون استفاده از این روش: (*pure store and forward*)

$$\text{end to end delay} = \text{number of links/hops} \times (t_{\text{prop}} + t_{\text{other}} + t_{\text{trans}})$$

با استفاده از این روش:

$$\text{end to end delay} = \text{number of links} \times (t_{\text{prop}} + t_{\text{other}}) + t_{\text{trans}}$$

در اسلاید بعد گفته که اگر دیتا را به پکت‌های بیشتر با سایز کوچک تر تقسیم کنیم، *delay* کمتری خواهیم داشت، اما چون به ازای هر پکت جدید، باز باید هدر را به آن بدهیم، پس اگر تعداد پکت ها زیاد شود و سایز بسیار کوچک شود، چون تعداد هدر ها به اندازه تعداد پکت ها افزایش یافته، همین باعث تاخیر بیشتر می‌شود. پس نقطه آپتیموم یک چیزی بین این دو است.

در صفحه بعدش، اسلاید ۳۸، تاخیر هاپ را ۲ تا در نظر گرفته (پس از تاخیر ۲ هاپ، اولین بیت به مقصد می‌رسد. این تعداد برابر با تعداد روتر های میانی ضربدر دیلی آنهاست احتمالا.) و از فرمول زیر برای محاسبه تاخیر ارسال استفاده کرده:

$$\text{end - to - end delay} = (\text{no. packets} + \text{no. hops}) \times (\text{size packet} + \text{size header})$$

(فرض کردیم که *propagation delay* نداریم.)

آن *no.hops* زمان تاخیر فرستادن اولین بیت پکت اول است.

زمان ارسال هر بیت/بایت را، اکتت تایم در نظر گرفته است.

برای پیدا کردن آپتیموم مثال اسلاید، از فرمول $(d + 2) \times (\frac{40}{d} + 3)$ مشتق می‌گیریم و مساوی صفر می‌گذاریم تا آپتیموم تعداد بسته را پیدا کنیم و احتمالا به عدد صحیح نزدیک تر گرد می‌کنیم.

تعداد پاکت ها برابر با اندازه *pipeline* در نظر گرفته شده بود که همه جا شوند.

اگر سرعت ورودی به لینک بیشتر از سرعت خط خروجی (*b/s*) باشد، پکت ها در روتر *queue* می‌شوند که به مدت زمان ماندن در صف *queueing delay* می‌گویند. اگر صف پر شود، بقیه پکت ها *lost* می‌شوند.

وظایف هسته شبکه:

- *forwarding*
- *routing* یا مسیریابی.

فورواردینگ یک عمل محلی است و توسط خود یک روتر انجام می‌شود اما روتینگ، در همکاری بین روترها صورت می‌گیرد.

طبق جدول فورواردینگ روتر که هر آدرس هدر خاص را به یک پورت، *map* کرده‌است، طبق آدرس موجود در هدر پکت، پورت خروجی آن تشخیص داده شده و به آن فوروارد می‌شود (در فیلد اول هدر پکت، آدرس مقصد آمده است).

شبکه‌های سویچینگ مداري *circuit switching*

در این جا مثل روش *vc* یک مدار بین مبدا و مقصد پیدا می‌شود، در فاز *connection setup*. اما برخلاف روش *vc*، این مدار اختصاصی است (حتی اگر برای مدتی داده‌ای برای ارسال نباشد). در واقع مدار بخشی از ریسورس‌های در مسیر را رزرو می‌کند و اطلاعات مربوط به آن اتصال را در هر روتر به صورت *state* ذخیره می‌شود. احتمالاً چون *state* داریم و مسیرها اختصاصی و مشخص است، نیازی به داشتن آدرس مقصد در هدر به همراه پکت نیست. بعد از آنکه داده ارسال شد، مسیر را پاک (*tear down*) می‌کنیم. منابع را آزاد می‌کنیم و استیت‌ها را پاک می‌کنیم. در شکل اسلاید، محور افقی مکان و محور عمودی زمان است. تاخیر ارسال را از روی ضخامت آبی می‌توان فهمید. تاخیر پردازش فقط در همان مرحله *connection setup* است که مربوط به پردازش روتر میانی برای یافتن مسیر و رزرو منابع و ذخیره حالت است.

مثلاً اگر بین دوتا روتر، ۴ تا مسیر باشد، ممکن است یکی از آن‌ها را رزرو کند.

روش‌های پیاده‌سازی برای سویچینگ مداري

• روش *FDM = frequency division multiplexing*:

در این روش، پهنای باند را به چند قسمت تقسیم می‌کنیم و هر کدام را به یک کاربر می‌دهیم. هر ۴ کاربر به بخش کوچکی از پهنای باند در همه زمان‌ها دسترسی دارد.

• روش *TDM = time division multiplexing*:

در این روش زمان را به فریم‌های زمانی تقسیم می‌کنیم و هر فریم را به ۴ اسلات تقسیم می‌کنیم. کاربر *i* ام به اسلات *i* ام از هر فریم دسترسی دارد و در آن مدت می‌تواند از کل پهنای باند استفاده کند.

در اسلایدها یک مسئله حل شد و مدت زمان ارسال داده در روش *FDM* و *TDM* هر دو یکی شد.

برای مقایسه *performance* دو روش سویچینگ بسته‌ای و سویچینگ مداري، باید به انواع تاخیرها، یعنی تاخیر *propagation* و *transmission data* و تاخیر گره توجه کرد.

یک مورد دیگر اینکه چون در *cs*، در همان مرحله *call setup*، مدار مشخص شده، شبکه برای داده *transparent* و شفاف است.

در اسلاید بعد گفته شد، هر فاز غیر از انتقال داده را، فاز *signalling* می‌گویند که مربوط به اطلاعات کنترلی است. دیتاگرام فقط ارسال داده را دارد و سیگنالینگ ندارد. از طرفی در *vc*، چون مسیر اختصاصی نیست، گپ‌هایی را در انتقال داده مشاهده می‌کنید (که این موضوع را در *cs* مشاهده نمی‌کنید. چون کانل اختصاصی است). مشکل *cs* این است که اگر کاربر، دیتایی نفرستد، چون لینک اختصاصی است، هیچ داده‌ای از آن عبور نمی‌کند و *utilization* خوبی ندارد، بر عکس *ps* که اگر یک یوزر، دیتا نفرستد، یوزرهای دیگر می‌توانند از آن مسیر، دیتاهای خود را بفرستند، *utilization* بسیار بیشتر است.

مقایسه بین *ps* و *cs*

روش *ps* برای داده‌های رگباری خوب است. (ترافیک‌های رگباری (*bursty*)، یعنی کاربران اغلب اوقات خاموش هستند و فقط بعضی اوقات، داده می‌فرستند). چون می‌تواند از مزیت اشتراک منابع استفاده کند. همچنین در این روش *call setup* اولیه را نداریم (پس روش *vc* چی؟!) و الگوریتم‌ها ساده‌تر هستند. از طرفی اگر شبکه دچار ازدحام *congestion* شوند، تاخیر شبکه زیاد می‌شود. برای مالتی مدیا و استریمینگ و تماس تلفنی، باید تاخیر کرانمند داشته باشیم که برای آنها *cs* روش بهتری است. پس اگر دنبال کارایی قابل پیش‌بینی و مطمئن و درستی ترتیب ارسال داده‌ها و راحتی فورواردیگ (به دلیل *exact match* و نبود سربار حافظه آدرس مقصد در هدر) هستیم، و یا می‌خواهیم داده‌ها یا ترافیک زیاد را همدل کنیم، باید به سراغ *cs* برویم.

در مقابل اگر داده‌ها کم است و می‌خواهیم از لینک‌ها با کارایی بالا استفاده کنیم و هزینه سربار ذخیره *state* در هر روتر یا *call setup* اولیه را ندهیم، بهتر است از *ps* استفاده کنیم. از آنجایی که *ps* فعلاً در شبکه خیلی رایج است، به گونه‌ای این مدل، *best effort* است، یعنی با اینکه هیچ تضمینی راجع به تاخیر و ترتیب، از بین رفتن داده و پهنای باند نمی‌کند، اما حداکثر تلاشش را می‌کند تا از هر طریق و لینک ممکن بسته‌ها را به مقصد برساند.

چند تا نکته راجع به روش *ps*:

اگر پکت، *loss* شد یا تاخیر خورد و یا *corrupt* شد، فرستنده دوباره ارسال می‌کند. اگر ترتیب داده‌ها به هم ریخته شده بود، گیرنده ترتیب را درست می‌کند. اگر شبکه *fail* شد یا ازدحام رخ داد، داده‌ها *drop* می‌شوند و از بین می‌روند.

مراجع

[1] “ch1-v8-slide-24-to-55,” <https://drive.google.com/drive/folders/>, 2020.